Cohortly
Software Engineering Major Project

Hamzah Ahmed

Contents

1. Definition and Analysis of Problem

1.1. Problem Statement and Project Proposal

Student cohorts perform the best when they collaborate. Stronger students in a subject can assist weaker students by sharing notes and doing peer study sessions. This also benefits the tutor, as it gives an opportunity for them to revise the material.

Currently, these sessions are uncoordinated and take place across disparate messaging applications. Study material, and answers to queries can easily get lost in these closed messaging channels, leaving out the opportunity for them to benefit others with the same questions.

A peer tutoring web-app would allow students to coordinate group study sessions, share notes and study material and have a shared set of questions and answers to post and view.

1.2. Stakeholders

1.2.1. Students

Such a system would be run most effectively when it is maintained primarily by students. Students will be both the tutors and the pupils in this system, where tutors benefit by revising the content by explaining it, and pupils benefit by catching up on the topics they may have missed in class. Additionally, some admin work is required to effectively run such a system, which should also be done by students. Student "moderators" volunteer to help keep a specific subject running smoothly by organising group sessions, and ensuring content is appropriate, correct and on-topic.

1.2.2. Teachers

A core requirement of this project is not to introduce any additional teacher workload. This system intentionally keeps teacher involvement minimal, as teacher involvement would not only create extra workload but would also affect group dynamics in a significant way. Additionally, teacher involvement would potentially create duty-of-care issues, where teachers may be liable to supervise student activity. Thus, the system does not involve teachers with any student-to-student interaction and teachers should only be able to see aggregated statistics on student progress and have access to a contribution leaderboard.

1.3. Specifications

1.3.1. Functional

Authentication

1. The system should allow teachers and students to log in using the school portal system
2. The system should allow students to:
    - Post Q&As
    - Schedule or join sessions
    - Share resources
    - Set peer tutoring availability
3. The system should allow moderators to:
    - Remove inappropriate or irrelevant content
    - Review flags for out-of-syllabus or disputed material
    - Add subtopics to subjects
4. The system should allow system administrators to:
    - Add or remove moderators
    - Create and manage subjects
5. The system should allow teachers only to:
    - View statistics on platform use (contribution credits) and student topic mastery ratings

Scheduling and Study Sessions

1. Students should be able to view public study sessions/tutoring sessions, and filter by:
    - Subject
    - Host (tutor or subject moderator)
    - Date/time
    - Sessions should show:
    - Subject or topic
    - Session title and description
    - Date and time
    - Meeting link or location
    - Host (student or moderator)
    - Participant list
    - Maximum capacity
2. Students should be able to:
    - Set availability as a peer tutor:
    - Set availability for tutoring sessions
    - Set size (max number of students) of session
    - Accept or decline requests
3. Subject moderators should be able to create general study sessions:
    - General study sessions timing is decided by scheduling polls
4. Students should be able to vote in scheduling polls:
    - Students vote for their available times, and the most popular time slot is chosen

Topic Hierarchy

1. The system should represent topics in a hiearchical way:
    - Topics are associated with a subject, and topics can have sub-topics corresponding to increasing detail
2. Students should be able to vote on the difficulty and mastery (how much they understand) of a topic
3. Teachers should be able to view aggregated statistics on difficulty and mastery metrics

Moderation

1. Students should be able to flag content as:
    - Inappropriate
    - Out of syllabus
    - Disputed
2. Moderators should be able to see a queue of flagged content, and choose an action (accept flag/delete/reject flag)
3. The system should store all moderator actions in an audit log, so moderator actions are held accountable to the moderator

Q&A

1. Students should be able to:
    - Post questions, associated with a topic in the topic hierarchy
    - Respond with answers
    - Flag a question

Resource Sharing

1. To share notes, worksheets and study materials, students should be able to:
    - Share links
    - Upload files
2. The system should tag resources with:
    - Subject
    - Topic
    - Uploader
    - Upload date
3. The system must scan uploaded files with VirusTotal API or similar
4. The system must limit to safe file formats (PDF, DOCX, PPTX, etc.)
5. Students should be able to flag resources

Points system

1. Students should be able to upvote resources, Q&A, and study sessions to indicate that they were helpful
2. After a study sessions, participants should be able to provide anonymous feedback on a study session, including:
    - A rating out of 5
    - General feedback
3. The system should provide students with **contribution credits** for participating, with higher credit amounts for more helpful activities
4. The system should display a leaderboard for the students with the highest credit amounts

1.3.2. Non-Functional

Usability

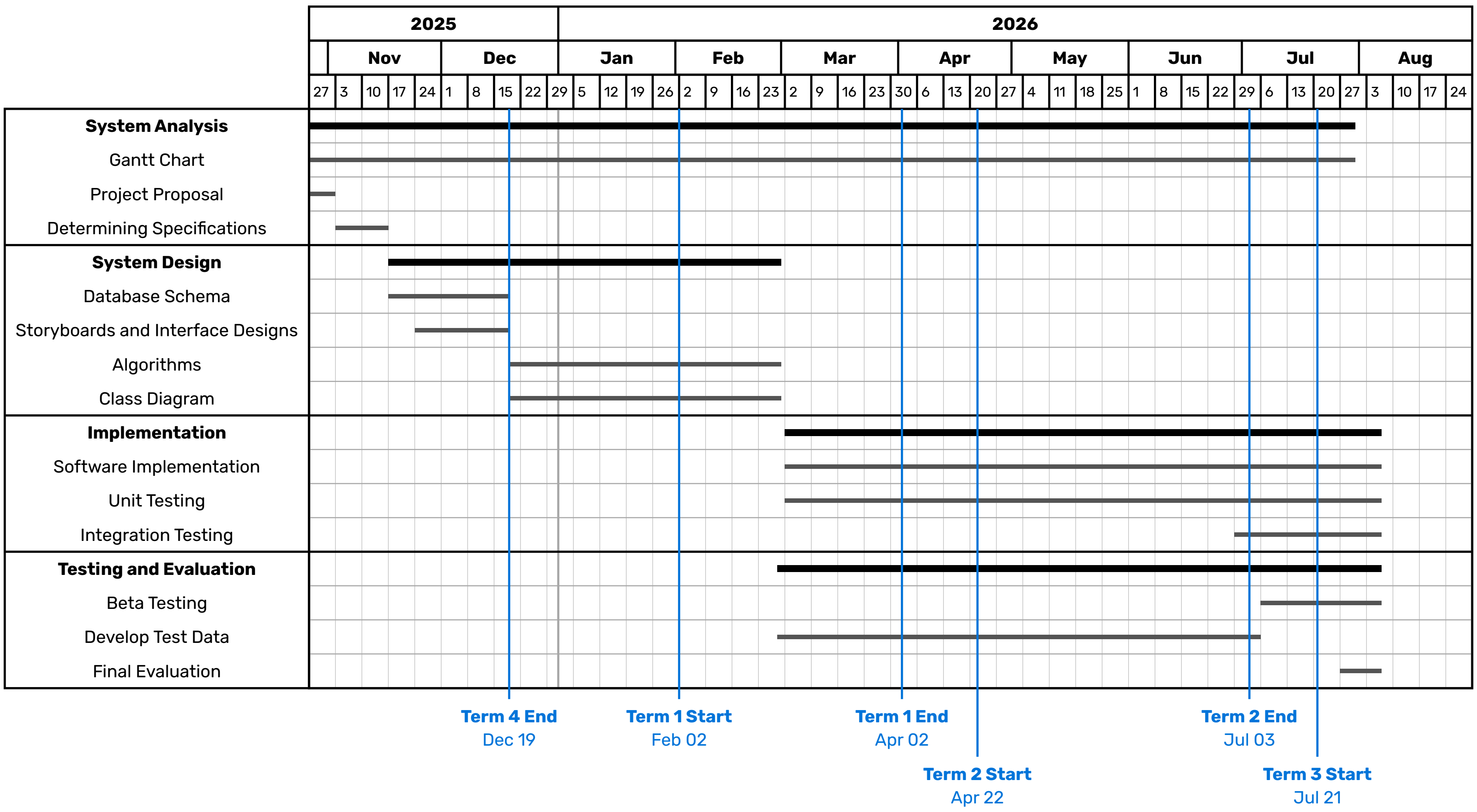
1. The system must be functional on desktop an mobile devices
2. The most commonly used actions must be easy to access intuitively
3. The interface must be consistent
4. There must be a help page to explain more complex processes (such as moderation, the contribution credits system, etc.)

Security and Privacy

1. Only students or teachers logged in through the school portal should be able to view any data within the system
2. Uploaded content should be automatically scanned with VirusTotal and restricted to safe formats

2. Research and Planning

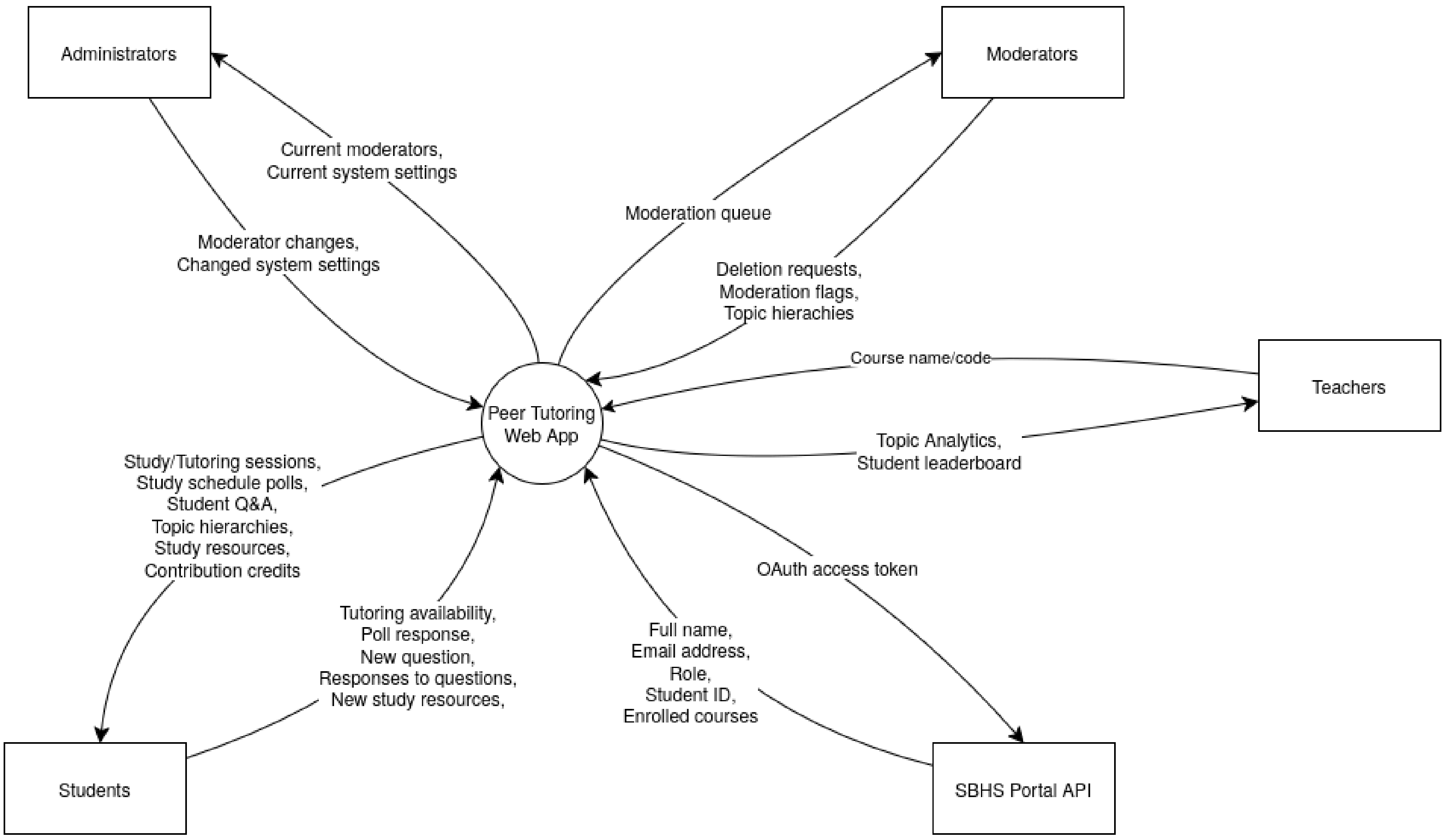
2.1. Gantt Chart



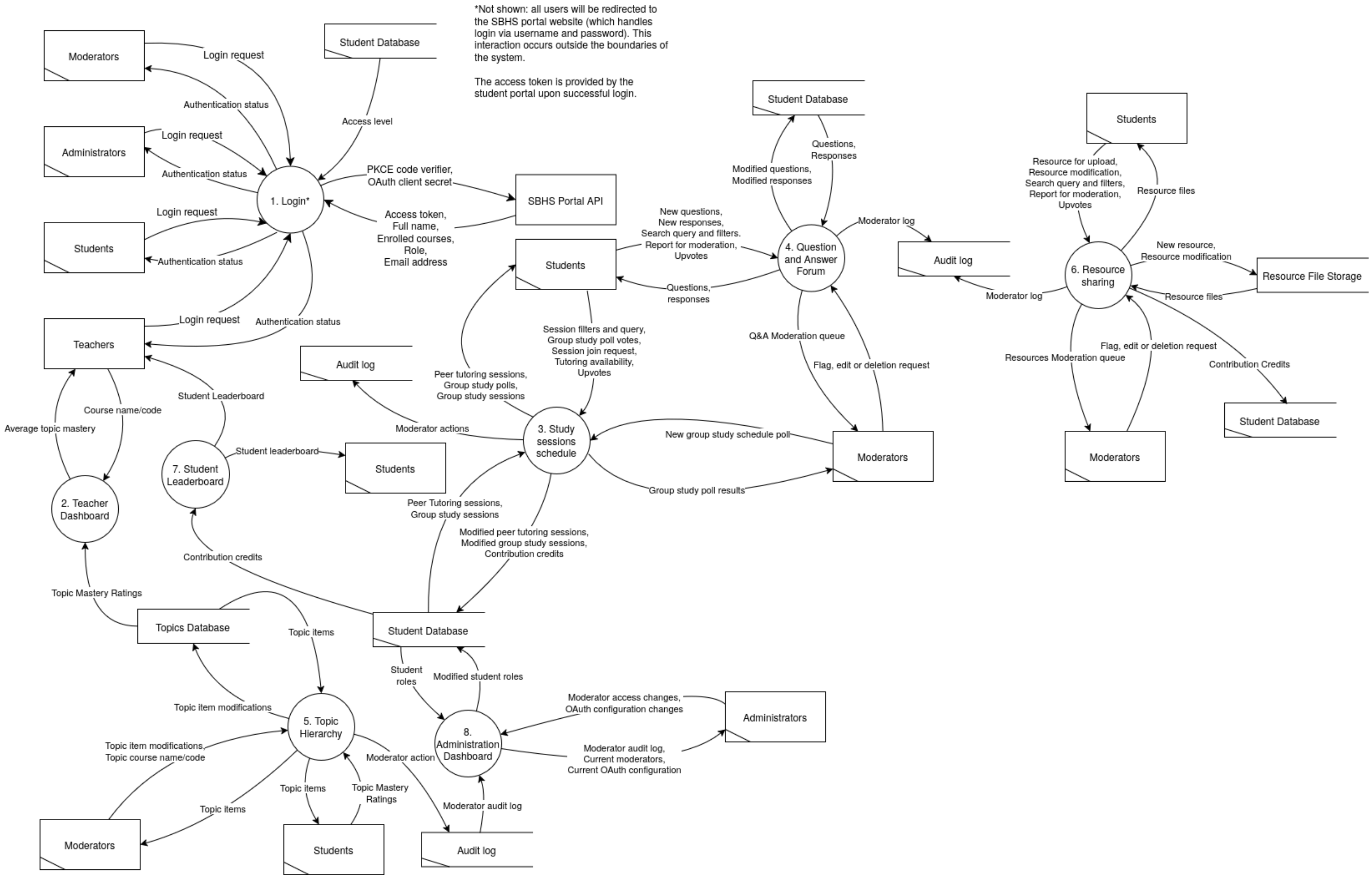
2.2. Design

2.2.1. Data flow Diagram

Context Diagram (Level 0)



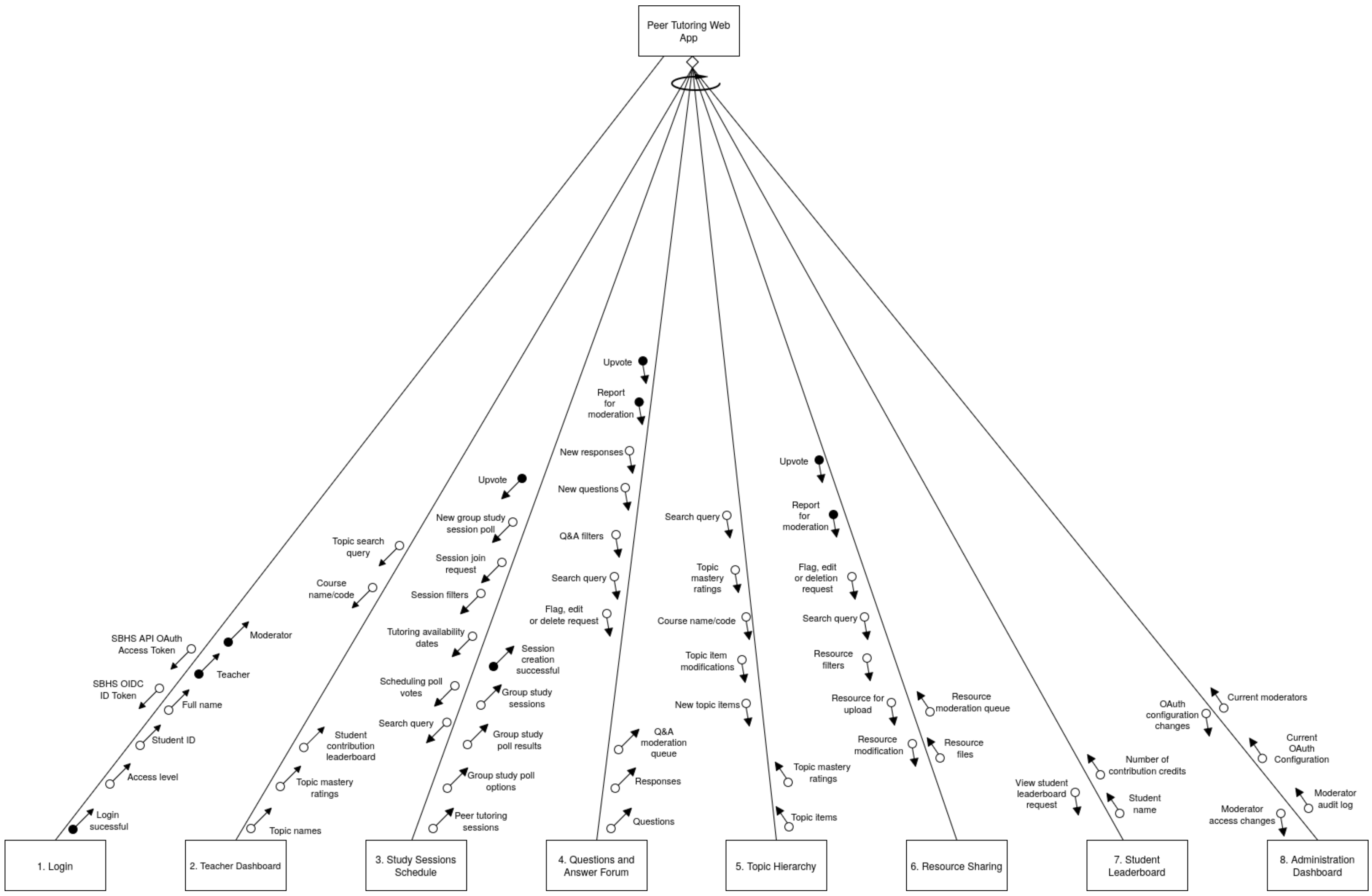
Level 1



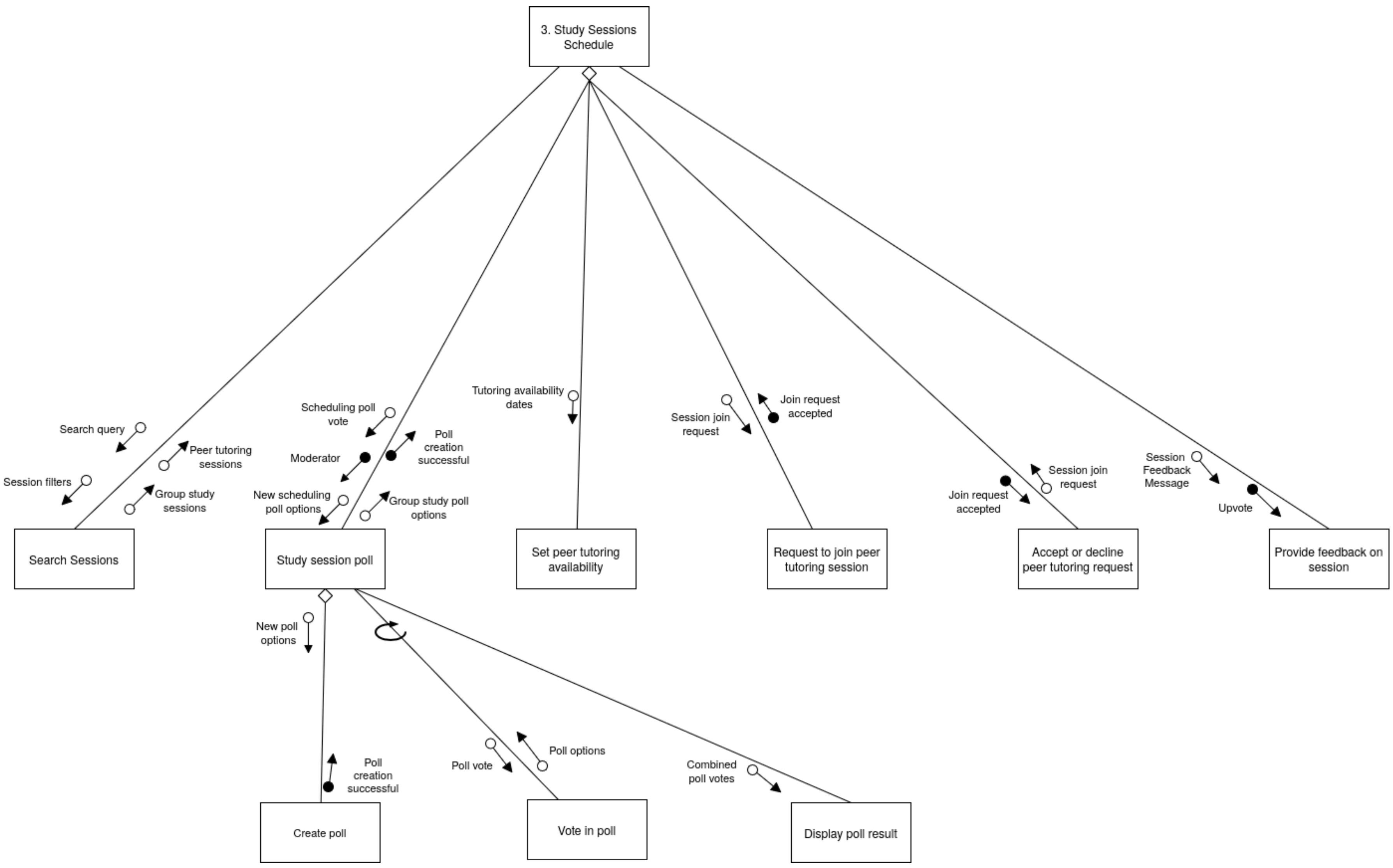
Level 2

2.2.2. Structure Chart

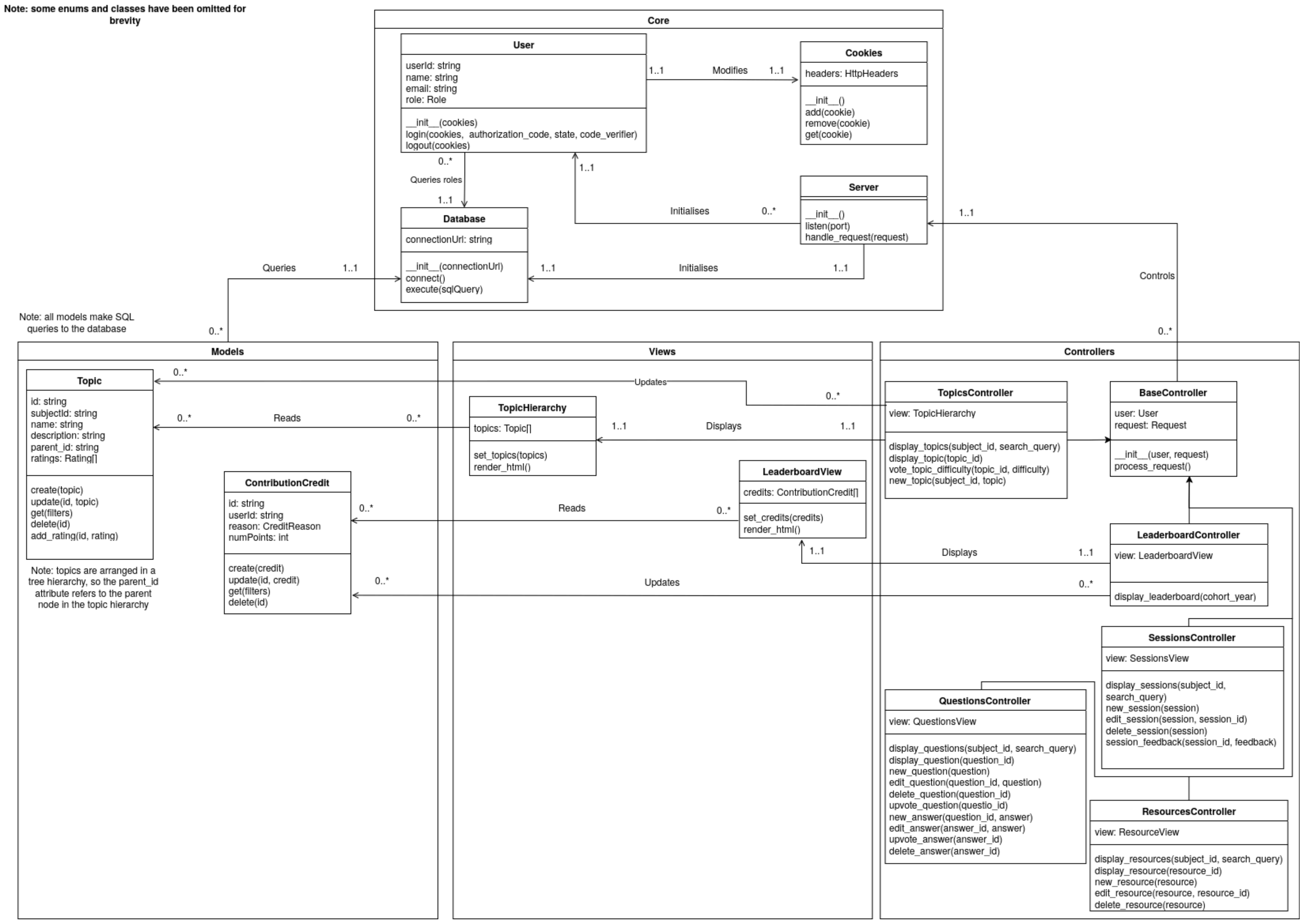
Level 1



Level 2

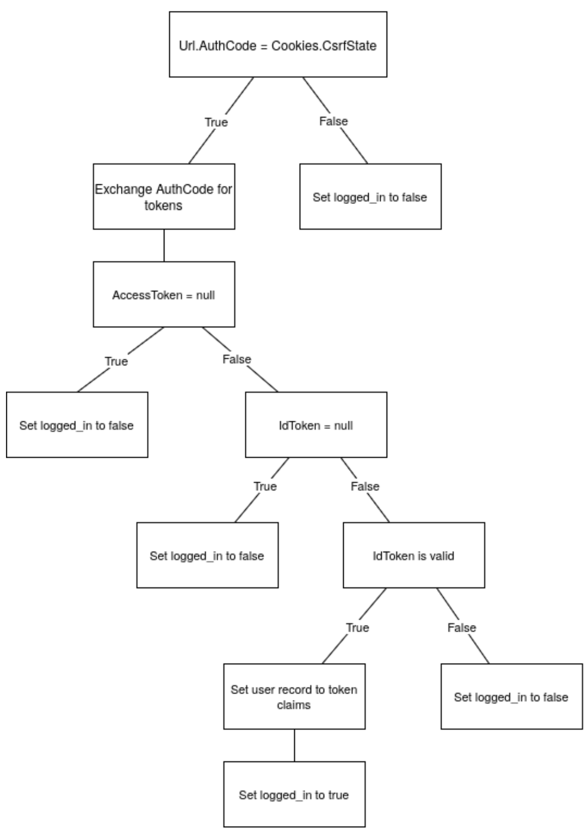


2.2.3. Class diagram

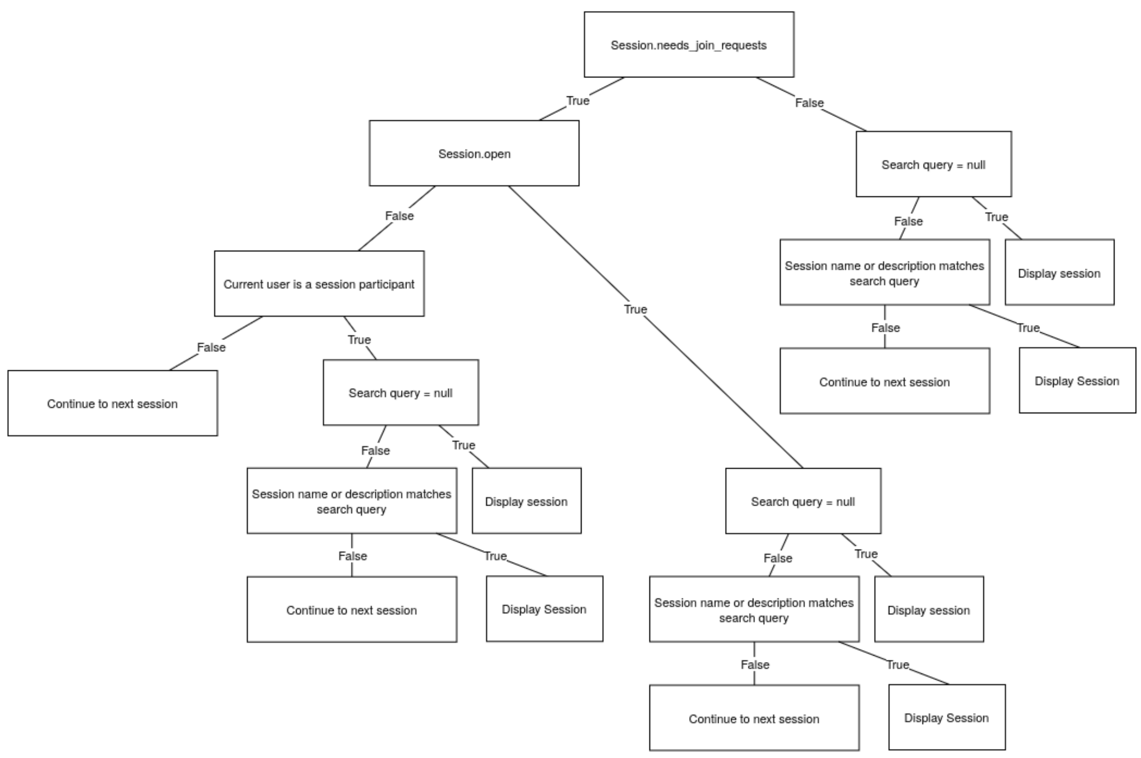


2.2.4. Decision Trees

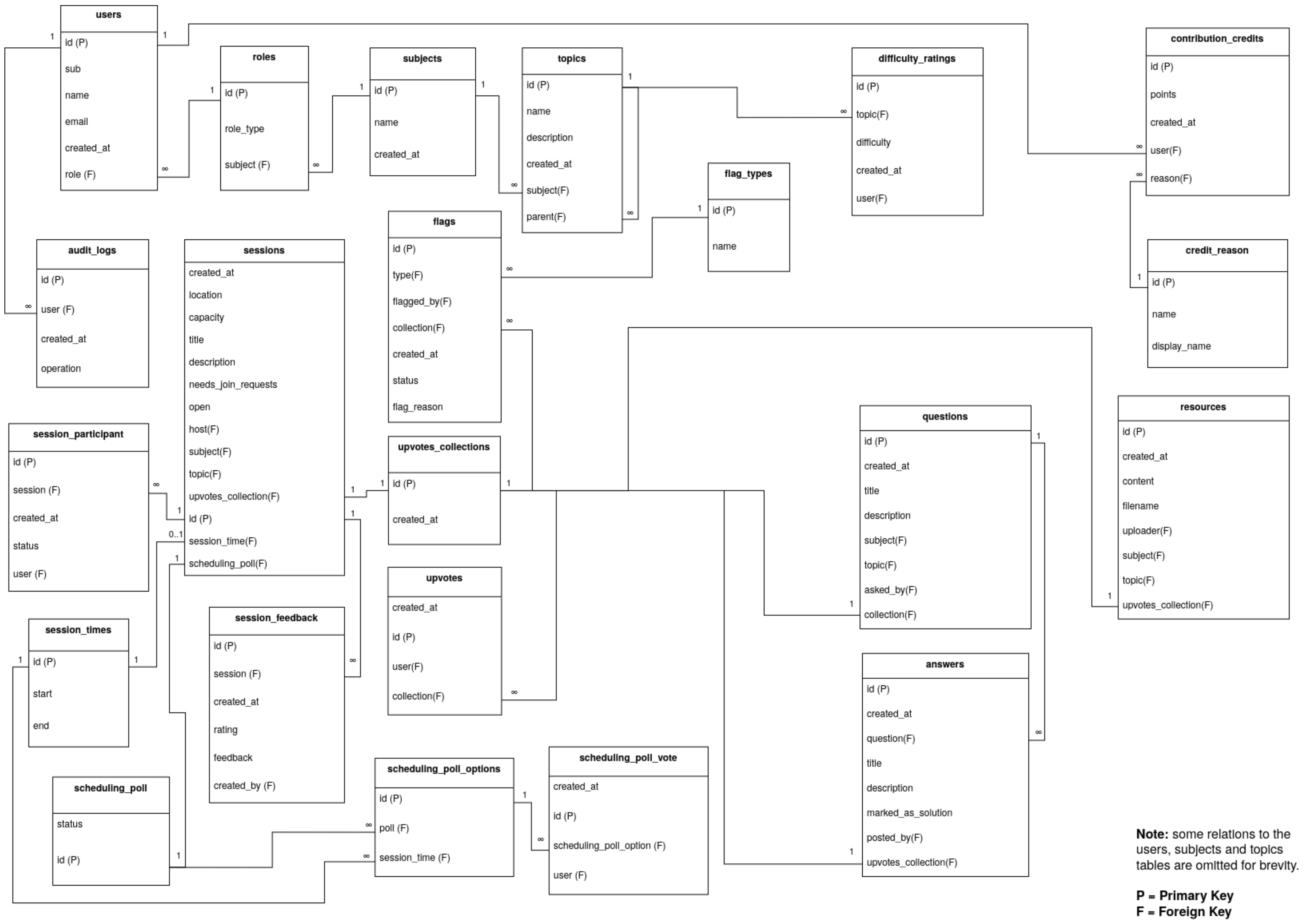
Login decision tree



Session search decision tree



2.2.5. Database Schema



2.2.6. Data Dictionary

Data Structure (type)	Attributes	Data type	Max length	Description
users (array of records)	id	integer	8 bytes	Primary key
	sub	string	100 chars	OIDC subject claim from school-portal supplied JWT
	name	string	70 chars	Full name of user (from school portal)
	email	string	254 chars	Email address of user (from school portal)
	created_at	timestamp	8 bytes	When the record was created
	role	integer	8 bytes	Role ID (foreign key to roles table)
	role_display	string	20 chars	Display name for role
	is_admin	boolean	1 byte	Whether the user has admin rights

Data Structure (type)	Attributes	Data type	Max length	Description
subjects (array of records)	id	integer	8 bytes	Primary key
	name	string	40 chars	
	created_at	timestamp	8 bytes	When the record was created
	topics	array of topic records	EOF	Subtopics of this subject

Data Structure (type)	Attributes	Data type	Max length	Description
topic (record)	id	integer	8 bytes	Primary key
	name	string	40 chars	
	description	string	EOF	
	created_at	timestamp	8 bytes	When the record was created
	subtopics	array of records	EOF	Subtopics of this topic

Data Structure (type)	Attributes	Data type	Max length	Description
difficulty_ratings (array of records)	id	integer	8 bytes	Primary key
	topic_id	integer	8 bytes	Foreign Key to topic
	created_at	timestamp	8 bytes	When the record was created
	user_id	integer	8 bytes	User who rated topic (foreign key to users)

Data Structure (type)	Attributes	Data type	Max length	Description
contribution_credits (array of records)	id	integer	8 bytes	Primary key
	points	integer	4 bytes	Number of points this contribution is worth
	created_at	timestamp	8 bytes	When the record was created
	user_id	integer	8 bytes	Recipient of the contribution credit (foreign key to users)
	reason_id	integer	8 bytes	ID of the credit reason (foreign key to credit_reasons)
	reason_name	string	20 chars	Internal name of credit reason
	reason_display_name	string	20 chars	Display name of credit reason

Data Structure (type)	Attributes	Data type	Max length	Description
audit_logs (array of records)	id	integer	8 bytes	Primary key
	user_id	integer	8 bytes	Moderator who performed the action (foreign key to users)
	created_at	integer	8 bytes	When the record was created
	operation	string	EOF	JSON description of the operation

Data Structure (type)	Attributes	Data type	Max length	Description

sessions (array of records)	created_at	integer	8 bytes	When the record was created
	location	string	100 chars	Voice channel or meeting link to join the session
	capacity	integer	4 bytes	Max number of students allowed to join the session
	title	string	30 chars	Title of the session
	description	string	EOF	Description of the session
	needs_join_requests	boolean	1 byte	Whether the session needs a request to join
	open	boolean	1 byte	Whether the session is open to new participants
	host	integer	8 bytes	User who hosts the session (foreign key to users)
	subject_id	integer	8 bytes	The subject the session is associated with (foreign key to subjects)
	topic_id	array of integer	EOF	Topics the session is associated with (foreign key to topics)
	id	integer	8 bytes	Primary key
	session_start	integer	8 bytes	Timestamp the session starts
	session_end	integer	8 bytes	Timestamp the session ends
	upvotes_id	integer	8 bytes	Foreign key to upvotes_collection
	scheduing_poll_id	integer	8 bytes	Foreign key to scheduling_poll
	participants	array of records	EOF	Participants in this session
	feedback	array of records	EOF	Feedback ratings given by session members

Data Structure (type)	**Attributes**	**Data type**	**Max length**	**Description**
participant (record)	id	integer	8 bytes	Primary key
	created_at	integer	8 bytes	When the record was created
	status_id	integer	8 bytes	The status of the students' request to join the session (foreign key to session_participant_statuses)
	status_display	string	20 chars	The display name for the status of the students' request to join the session
	user_id	integer	8 bytes	The participant (foreign key to users)

Data Structure (type)	**Attributes**	**Data type**	**Max length**	**Description**
feedback (record)	id	integer	8 bytes	Primary key
	created_at	integer	8 bytes	When the record was created
	rating	integer	4 bytes	Rating out of 5 for the session
	feedback	string	EOF	Optional written feedback for the session
	created_by	integer	8 bytes	Who created this feedback record (foreign key to user)

Notes:

- Character lengths are provisional lengths for the size of a single line when displayed.
- In SQLite, the max length of an `INTEGER` is 8 bytes, so that length is used for all IDs
- 8 byte Unix epoch timestamps are used, as 4 byte epoch timestamps cannot represent timestamps past 19 January 2038

2.2.7. Algorithms

Main program

```
logged_in = false
User = Null

BEGINMAIN
   WHILE logged_in = false
```

```
    User = Login()
   ENDWHILE

   WHILE logged_in
      Get user input
      CASEWHERE user input
        'admin'         : AdminPage()
        'leaderboard'   : LeaderboardPage()
        'sessions'      : SessionsPage()
        'questions'     : QuestionsPage()
        'resources'     : ResourcesPage()
        'topics'        : TopicsPage()
        'log out'       : logged_in = false
      END CASE
   ENDWHILE
ENDMAIN
```

Login

```
BEGIN Login()
    CsrfToken = GenerateRandomString()

    RedirectToSbhsPortal()

    AuthCode = GetParamFromUrl('code')
    CsrfState = GetParamFromUrl('state')

    IF AuthCode <> CsrfState THEN
        logged_in = false
        RETURN Null
    ENDIF

    Response = ExchangeAuthCodeForTokens()

    AccessToken = GetFromResponse(Response, 'access_token')
    RefreshToken = GetFromResponse(Response, 'refresh_token')
    IdToken = GetFromResponse(Response, 'id_token')

    IF AccessToken = Null THEN
        logged_in = false
        RETURN Null
    ENDIF

    IF IdToken <> Null THEN
        TokenValid = VerifyToken(IdToken)       'Note that this uses digital signature algorithms (DSA) to verify the token
rather than a database search
        IF TokenValid THEN
            Claims = GetUserClaims(IdToken)
            logged_in = true
            RETURN CreateUserRecord(Claims.id, Claims.name, Claims.email)
        ENDIF
    ENDIF

    RETURN Null
END Login
```

Leaderboard

```
BEGIN LeaderboardPage()
    CohortYear = Get user input

    LeaderboardRecords = []

    Open StudentDatabase
    Open CreditsDatabase

    FOR i = 0 to StudentDatabase.Length
        StudentRecord = StudentDatabase[i]

        'This is implemented using an SQL WHERE clause
        CreditRecords = FilterByUserId(CreditsDatabase, StudentRecord.UserId)

        Points = 0
        For j = 0 to CreditRecords.Length
            Points = Points + CreditRecords[j].Points
        NEXT j

        LeaderboardRecord = Empty leaderboard record
        LeaderboardRecord.Student = StudentRecord
        LeaderboardRecord.Points = Points
    NEXT i

    MergeSortDescendingByPoints(LeaderboardRecord)

    Close CreditsDatabase
    Close StudentDatabase

    FOR i = 0 to 29
        IF i < LeaderboardRecords.Length THEN
            Record = LeaderboardRecords[i]
            Display 'Place', i + 1
            Display 'Name', Record.Student.Name
            Display 'Points', Points
        ENDIF
    NEXT i
END LeaderboardPage

BEGIN MergeSortDescendingByPoints(Records)
    IF Records.length <= 1 THEN
        RETURN Records
    ENDIF

    Middle = Records.length / 2

    Left = MergeSortDescendingByPoints(Records from index 0 to Middle)
    Right = MergeSortDescendingByPoints(Records from index Middle + 1 to Records.length - 1)

    RETURN MergeByPointsDescending(Left, Right)
END MergeSortByPoints

BEGIN MergeByPointsDescending(Left, Right)
    MergedArray = []
    WHILE Left is not empty AND RIGHT is not empty
        IF Left.Length > 0 AND Right.Length > 0 THEN
            IF Left[0].Points > Right[0].Points THEN
                Append Left[0] to MergedArray
                Remove Left[0]
            ELSE
                Append Right[0] to MergedArray
                Remove Right[0]
            ENDIF
        ELSE IF Left.Length > 0 THEN
                Append Left[0] to MergedArray
                Remove Left[0]
        ELSE IF Right.Length > 0 THEN
                Append Right[0] to MergedArray
                Remove Right[0]
        ENDIF
    ENDWHILE

    RETURN MergedArray
END MergeByPointsDescending
```

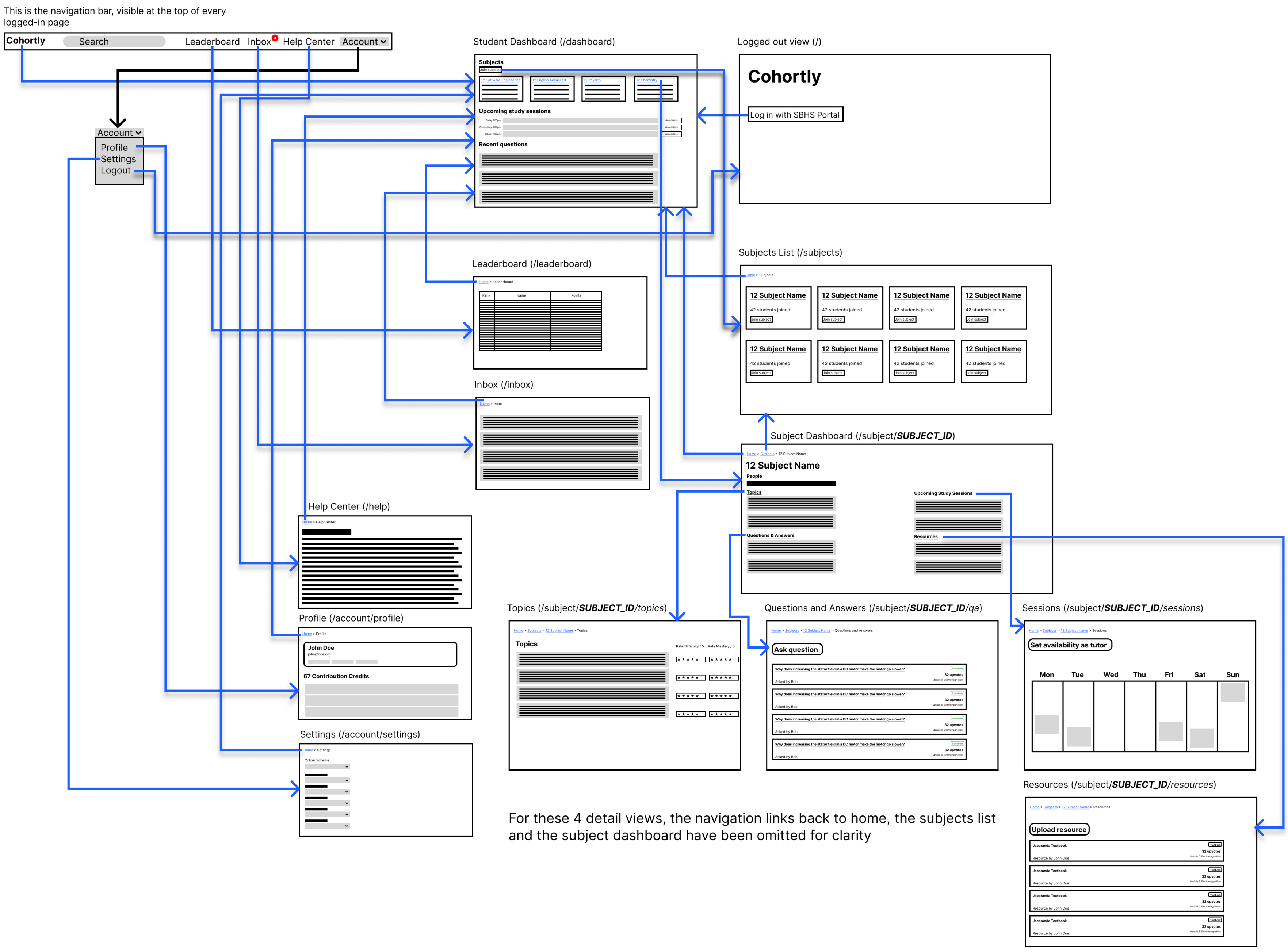
Questions Page

```
BEGIN QuestionsPage()
    Get user input
    CASEWHERE user input
        'search_questions'  :  QuestionsSearch()
        'new_question'      :  NewQuestion()
        'edit_question'     :  EditQuestion()
        'delete_question'   :  DeleteQuestion()
        'upvote_question'   :  UpvoteQuestion()
        'new_answer'        :  NewAnswer()
        'edit_answer'       :  EditAnswer()
        'delete_answer'     :  DeleteAnswer()
        'upvote_answer'     :  UpvoteAnswer()
    END CASE
END QuestionsPage
```

Session Page

```
BEGIN SessionsPage()
    Get user input

    CASEWHERE user input
        'search'             : SessionsSearch()
        'group_polls'        : SessionsPoll()
        'peer_tutoring'      : SessionsPeerTutoring()
        'feedback'           : SessionsFeedback()
    END CASE
END SessionsPage

BEGIN SessionsSearch()
    Get user input
    Open SessionsDatabase

    Index = 0
    WHILE Index < SessionsDatabase.Length
        IF Contains(SessionsDatabase[Index].name, user input) OR
           Contains(SessionsDatabase[Index].description, user input) THEN  'Note that the implementation will use a fuzzy
search algorithm for more relevant search results
            Display 'Name', SessionsDatabase[Index].name
            Display 'Description', SessionsDatabase[Index].description
            Display 'Subject', SessionsDatabase[Index].subject
            Display 'Date', SessionsDatabase[Index].date
            Display 'Time', SessionsDatabase[Index].time
            Display 'Location', SessionsDatabase[Index].location
            Display 'Host', SessionsDatabase[Index].host
        ENDIF
        Index = Index + 1
    ENDWHILE

    Close SessionsDatabase
END SessionsSearch
```

2.2.8. Storyboard



Zoom for detail.

2.2.9. Interface Design

Consistency

Responsive Web Design

Navigation

Grouping

Effective use of colour

Accessibility

3. Producing and Implementing

3.1. Logbook

3.1.1. 16 November 2025

At this stage I have some of my early specifications, so I can start considering what technologies to use for implementation:

Since my project is collaborative, functionality has a heavy dependency on the shared database. Most of the content in the web app is located on the server, so there is not much point having a JS-first application "shell" syncing server side state, since the shell can't do much on its own. Thus a server-first implementation, with content rendered to HTML on the server makes the most sense. Additionally, offline functionality can still be available through read-only service worker caching of rendered pages.

Technology stack: Rust-based server side libraries: Axum for server Maude for templating (html generated by rust macros) This allows for a JSX-like component system except its entirely server-side Lightweight client side libraries: Alpine.js for simple client side interactivity Possibly solid or svelte for any highly interactive component islands (but loaded only in those places) HTMX for no-reload form and link UX Pico css for semantic html styling

Authentication: Server-side OAuth: students login with student portal, their ID token, access token, refresh token are all stored in HTTP-only cookie

3.1.2. 7 January 2026

Completed authentication middleware: Can now log in with the SBHS portal, interfacing with OAuth and OIDC The middleware extracts name, user ID and email address from the ID Token (JWT)

3.1.3. 21 April 2026

Completed database integration. I chose SQLite for the database for its ease of deployment. SQLite is a bit more bare in functionality compared to some heavier databases and suffers from relaxed typing, but using it means that the system can be deployed as a single container with a persistent volume attached. If this ends up being student-run that is very useful. Can now create courses in the database which students can join I chose breadcrumb navigation to have a consistent way for users to navigate the hierarchical interface.

3.1.4. 12 July 2026

Finished implementation of subjects and topics in the peer tutoring system.

3.1.5. 18 July 2026

Began rewrite of major project to Python with Django. The axum-based stack proved unproductive since the project required many repetitive forms. This proved far more efficient in Django, compared to axum where the form HTML must be written manually.

Most of the HTML and styles could be carried over without too much change, so much of the work went into backend improvement. Despite a slight initial learning curve, after very few hours this approach proved very productive.

3.1.6. 19 July 2026

Completed implementation of all existing features in the new Django-based system. Including:
- Authentication
- Subjects
- Initial topics system

3.1.7. 21 August 2026

Finished topic system, allowing moderators to create, edit and reorder topics. Topic descriptions are markdown text, allowing for features such as headings, code and images to be included in the description. This markdown system was planned to be reused in other areas of the web application, allowing for rich text in questions and answers.

3.1.8. 23 August 2026

Completed question and answer system, including upvoting questions and answers, and marking an answer as a solution. Also created a search feature, re-implemented profile and settings menus, and much of the initial study sessions scaffolding.

3.1.9. 24 August 2026

Implemented interactive calendars using the `fullcalendar.js` library, supporting efficient visualisation of study sessions. A similar calendar was also included grade-wide to allow the prevention of scheduling conflicts between different subjects.

3.1.10. 26 August 2026

Completed folio documentation.

3.2. User documentation

This is the user documentation included in the product.

3.2.1. Getting Started

Cohortly is a web application designed to foster student collaboration by allowing students to share their knowledge in Q&As and study sessions.

To use Cohortly, you log in with your School Portal account. Your account is linked to your email address, student ID and full name. Access is limited to Sydney Boys High School students only. Once logged in, you will be redirected to the Dashboard.

3.2.2. Joining a subject

From the dashboard, you can join a subject by clicking the “Join or leave subjects” button. From there you can manage your subject memberships. Joining a subject is required before you can use the study sessions or Q&A features. From there, you can click on a subject to view its detail page.

3.2.3. Asking and answering questions

From the subject page, you can click on the “Questions” link in the sidebar to access the questions list. From there, you can ask a question, or view existing questions. Within a question, you can upvote the question if you find it helpful, post an answer, and upvote answers. Upvoted questions and answers appear earlier in their lists, allowing people to find them more easily. If you posted a questions, you can mark the answer that best answers it as the solution to your question, where it will appear at the top of the answers list to help people find it more easily. Since questions serve as a resource for other students, they should be clear and well-written, asking a specific syllabus-relevant question.

3.2.4. Joining study sessions

On your dashboard, you can see the study sessions for all subjects across the Cohortly instance for your grade. Clicking on any entry will send you to the session detail page for that session, showing description, time, location, participants and other relevant information. If you are a member of that subject, you can join the session by clicking on “Join session” on the session detail page. Sessions filtered to a subject can be accessed by clicking on “Sessions” in the sidebar on any subject page.

3.2.5. Frequently Asked Questions

Why can’t I join a session?

Sessions are capacity limited to a number of students decided by the moderator. This prevents groups from becoming too large and making it harder to support students. Session participation is assigned on a first-come-first-serve basis. When a session is at capacity, the join button will be grayed out.

Why can’t I see my subject on the dashboard?</h2>
Your dashboard will only show a subject you are a member of. If you have not yet joined a subject it will not be visible on that page. If your subject is not visible on the subjects list page, then you will need to contact the administrator to add the subject.

How are new subject requests processed?
Before a new subject is created, a moderator who will manage group study sessions and Q&A content needs to be appointed. More than one moderator can be appointed for large subjects to split the workload. Moderator choice is subject to the administrator’s discretion, and moderator privileges can be revoked if abused. A moderator can only moderate content for their appointed subject.

How does moderation work?
Moderators chosen for each subject by an administrator can create and modify group study sessions. They can also modify or delete questions and answers if they contain inappropriate content.

4. Testing and Evaluating

4.1. Test Report

4.1.1. Authentication

This is handled by the student portal.

Input (username)	Input (password)	Expected output	Reason
4440000000	Correct corresponding password	The app loads student full name and email address, and the main dashboard is shown.	Testing successful authentication
bob	banana	Student portal displays error message and does not redirect the user to the web app	Testing non-existent accounts

4.2. Reccomendations

This system passes preliminary testing of the main cases, as well as some user inputs designed to test input santisiation and resistance to XSS attacks. However, this testing methodology lacks more sophisticated testing such as load testing, automated fuzzing (DAST), and testing for race conditions. These tests should be perfomred before installation of the system.

Appendix A. Summary of survey results